

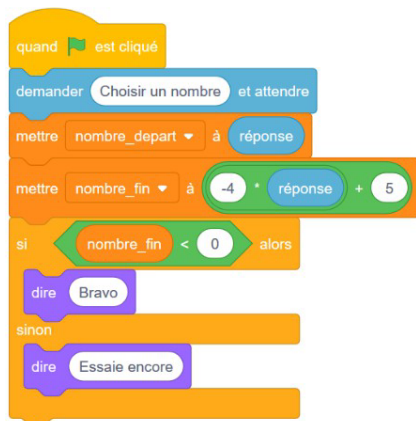
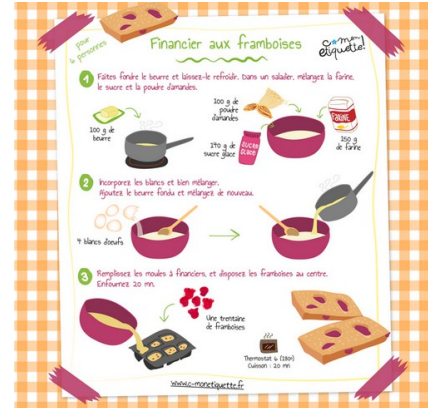
Cours	Comprendre l'architecture d'un programme Python				CIEL-IR
Maths-Info	Activités	R2 ; D5	Compétences	C08 ; C09 ; C10	Date :

1. Une analogie simple pour démarrer

Un programme informatique peut être comparé à une **recette de cuisine** :

- Les **ingrédients** représentent les données (variables) avec lesquelles on travaille.
- Les **étapes de la recette** sont les instructions ou le code qui transforment ces données.
- Le **plat final** est le résultat ou la sortie du programme.

Sans organisation, une recette devient **confuse** ou **infaisable**, tout comme un programme mal structuré devient lui aussi difficile à **comprendre**, à **utiliser** ou à **corriger**.



```
nombre_depart = float(input("Choisir un nombre : "))
nombre_fin = -4 * nombre_depart + 5
if nombre_fin < 0:
    print("Bravo")
else:
    print("Essaie encore")
```

2. Les bases d'un programme

Pour structurer un programme, il faut comprendre ses composantes fondamentales :

2.1. Les données : les variables

Une variable est une "boîte" où l'on stocke une information. Elle donne un nom à cette donnée pour qu'on puisse la réutiliser.

Exemple :

```
nom = "Alice"
age = 25
```

"On a une boîte appelée **nom** contenant 'Alice' et une autre appelée **age** contenant 25."



2.2. Les instructions : la logique

Les **instructions** sont les étapes que suit l'ordinateur pour manipuler ou transformer les données. Cela inclut des **décisions (conditions)** et des **répétitions (boucles)**.

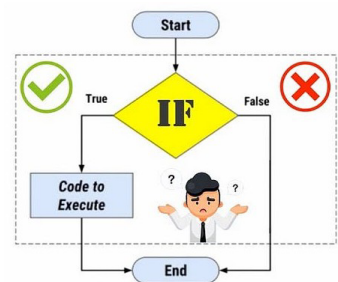
2.2.1. Condition : prendre une décision.

```

if age >= 18:
    print("Vous êtes majeur.")
else:
    print("Vous êtes mineur.")

```

"Si l'âge est supérieur ou égal à 18,
on affiche : 'Vous êtes majeur',
sinon
on affiche : 'Vous êtes mineur'."



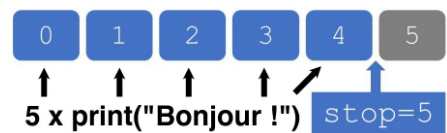
2.2.2. Boucle : répéter une action
un exemple avec range()

```

for i in range(5):
    print("Bonjour !")

```

On répète « Bonjour ! » cinq fois,
puis la boucle s'arrête.



2.3. Les fonctions : organiser et réutiliser

Une fonction est une **machine** que l'on construit pour accomplir une tâche précise. Elle permet de regrouper du code et de le réutiliser facilement.



Exemple :

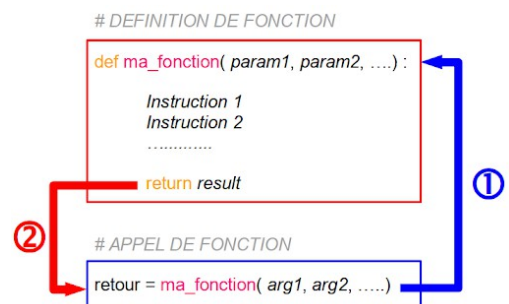
```

def saluer(nom):
    print(f"Bonjour, {nom} !")

saluer("Alice")
saluer("Bob")

```

"On crée une machine appelée **saluer** qui dit 'Bonjour' à la personne indiquée. On peut l'utiliser autant de fois qu'on veut."



3. Pourquoi une architecture est essentielle

Un programme bien structuré :

- Est **facile à comprendre**, même par une autre personne.
- Permet d'**éviter les erreurs**, car chaque partie a un rôle précis.
- Est **modulable** et **réutilisable**, ce qui facilite les ajustements.



3.1. Exemple d'une mauvaise architecture

Voici un programme désorganisé pour additionner deux nombres :

```
a = 3
b = 5

if True:
    print(a + b)
else:
    a + b # Inutile
```

Problèmes :

1. **Pas de fonction** : Impossible de réutiliser ce code sans le recopier.
2. **Condition inutile** : Le **if True** est toujours vrai, donc l'**else** ne sert à rien.
3. **Manque de clarté** : Pourquoi **a** et **b** existent n'est pas évident.



3.2. Exemple d'une bonne architecture claire

Voici une version bien structurée du même programme :

```
def addition(a, b):
    return a + b

resultat = addition(3, 5)
print("La somme est :", resultat)
```

Pourquoi c'est mieux ?

1. **Fonction réutilisable** : La fonction **addition()** peut être appelée avec d'autres nombres.
2. **Clarté** : Chaque partie a un rôle précis (calcul, stockage, affichage).
3. **Facile à modifier** : Si on veut multiplier au lieu d'additionner, il suffit de modifier la fonction.



4. Les concepts clés pour une bonne architecture

4.1. Pourquoi déclarer les variables avant de les utiliser ?

L'ordinateur ne peut pas deviner ce qu'une variable représente. Si elle n'est pas déclarée, Python génère une erreur.

Exemple :

```
x = 5 # OK
print(x) # Affiche 5

print(y) # Erreur : `y` n'a pas été défini.
```

4.2. Pourquoi utiliser **return** ?

return sert à renvoyer un résultat d'une fonction pour qu'il soit utilisé ailleurs.
Sans **return**, un résultat est "bloqué" dans la fonction et inutilisable en dehors.

Exemple :

```
def addition(a, b):
    return a + b

resultat = addition(2, 4)
# `resultat` contient maintenant 6
```

4.3. Pourquoi un **if** sans condition est inutile ?

Un **if** sans vraie comparaison complique le code inutilement.

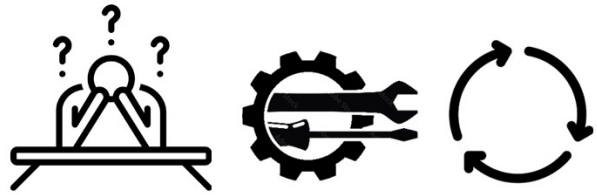
Dans le code ci-contre, le **if** n'a aucun intérêt, car il est toujours vrai.

Exemple inutile :

```
if True:
    print("Toujours vrai.")
else:
    print("Jamais exécuté.")
```

5. Les pièges d'une mauvaise architecture

- **Difficultés à comprendre** : Les autres (ou vous-même plus tard) ne comprendront pas le but du programme.
- **Problèmes de maintenance** : Sans information, la modification un détail risque d'introduire des erreurs.
- **Manque de ré-utilisabilité** : Vous devrez réécrire le code au lieu de simplement appeler une fonction.



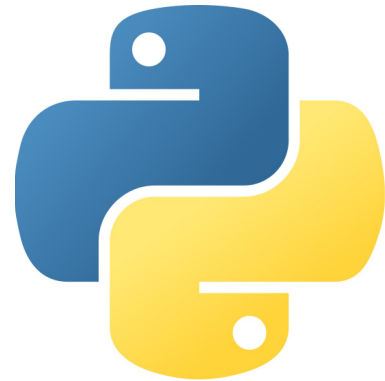
6. Conclusion

L'architecture d'un programme, et pas uniquement en PYTHON, c'est **son plan, sa structure, son but...**

Cette architecture permet de :

- Créer des programmes **lisibles, clairs et logiques**.
- **Éviter les erreurs** et faciliter les modifications.
- **Réutiliser** et **partager** du code efficacement.

Avec cette base, même un néophyte peut comprendre pourquoi une bonne structure est essentielle et comment l'appliquer en Python.



Enfin, la règle d'or pour une programmation réussie est :

"Est-ce que je peux expliquer chaque partie facilement ?"

Si la réponse est "NON", le programme manque probablement d'une bonne architecture.

